Avance final del portafolio de evidencias

CURSO

Programación Orientada a Objetos

Fecha

26/04/2026

Estudiantes:

Yen Lip Lee Espinoza

Profesor:

ROMARIO SALAS CERDAS

Tabla de contenido

Portada 1
Tabla de contenido **2**
Introducción **3**
Explicación del Sistema y del Diseño **3**
Manejo de Excepciones **4**
Arquitectura del Sistema **4**
Persistencia de Datos **5**
Repositorio del Proyecto **5**
Diagrama UML **5**
Conclusión **7**
Conclusión **9**
Referencias **10**

Introducción

El presente proyecto consiste en el desarrollo de un Sistema de Reservas Hoteleras implementado en el lenguaje de programación Java, aplicando los principios fundamentales de la Programación Orientada a Objetos (POO). El objetivo principal es diseñar y construir un sistema funcional que permita la gestión de clientes, habitaciones y reservas, integrando persistencia de datos mediante una base de datos MySQL.

Este sistema fue desarrollado siguiendo una arquitectura por capas, separando claramente la lógica del negocio, el acceso a datos y la interfaz de usuario. Además, se implementaron conceptos clave como abstracción, encapsulamiento, herencia, interfaces y manejo de excepciones, con el fin de lograr un diseño modular, mantenible y escalable.

El proyecto también incluye la aplicación del patrón DAO (Data Access Object) para la interacción con la base de datos, permitiendo una adecuada separación de responsabilidades y facilitando futuras mejoras o cambios en la tecnología de persistencia.

Explicación del Sistema y del Diseño

El sistema desarrollado permite realizar las siguientes operaciones:

- Registro de clientes
- Registro de habitaciones
- Creación de reservas
- Visualización de información almacenada
- Cancelación de reservas
- Persistencia de datos en base de datos MySQL
- Aplicación de Programación Orientada a Objetos

Durante el desarrollo del sistema se aplicaron los siguientes conceptos:

Abstracción

Se implementa mediante la clase abstracta Persona, la cual define atributos y comportamientos comunes para diferentes tipos de usuarios del sistema, sin permitir su instanciación directa.

Encapsulamiento

Todos los atributos de las clases se definen como privados, accediendo a ellos mediante métodos públicos (getters y setters). Esto protege la integridad de los datos y permite controlar su modificación.

Herencia

Las clases Cliente y Administrador heredan de la clase Persona, reutilizando sus atributos y métodos, evitando duplicación de código y promoviendo la reutilización.

Interfaces

Se implementa la interfaz Imprimible, la cual define el método generarResumen(). Esta interfaz es implementada por las clases Cliente, Habitacion y Reserva, permitiendo aplicar polimorfismo y estandarizar la forma en que los objetos muestran su información.

Composición

La clase Reserva contiene una instancia de Habitacion, lo que representa una relación fuerte donde la reserva depende directamente de la habitación asociada.

Agregación

La clase Cliente mantiene una colección de reservas, lo que representa una relación más débil donde un cliente puede tener múltiples reservas.

Manejo de Excepciones

Se implementaron excepciones personalizadas como:

- ClienteNoEncontradoException
- HabitacionNoDisponibleException
- ReservaNoEncontradaException
- DatoInvalidoException

Esto permite manejar errores de manera controlada y mejorar la robustez del sistema.

Arquitectura del Sistema

El sistema fue desarrollado utilizando una arquitectura por capas, organizada de la siguiente manera:

- UI (Interfaz de Usuario): Maneja la interacción con el usuario mediante consola (Menu, Main)
- TL (Lógica de Transferencia): Controlador que conecta la UI con la lógica (Controlador)
- BL (Business Logic): Contiene la lógica del negocio (Gestores)
- DAO: Encargado del acceso a la base de datos
- DL (Data Layer): Manejo de conexión a la base de datos (Conector, AccesoBD)
- Utils: Lectura de configuraciones (bd.properties)

Esta estructura permite una mejor organización del código, facilitando su mantenimiento y escalabilidad.

Persistencia de Datos

El sistema utiliza una base de datos MySQL para almacenar la información de:

- Clientes (t_clientes)
- Habitaciones (t_habitaciones)
- Reservas (t_reservas)

La conexión se realiza mediante JDBC, utilizando un archivo de configuración (bd.properties) para manejar las credenciales de acceso.

Repositorio del Proyecto

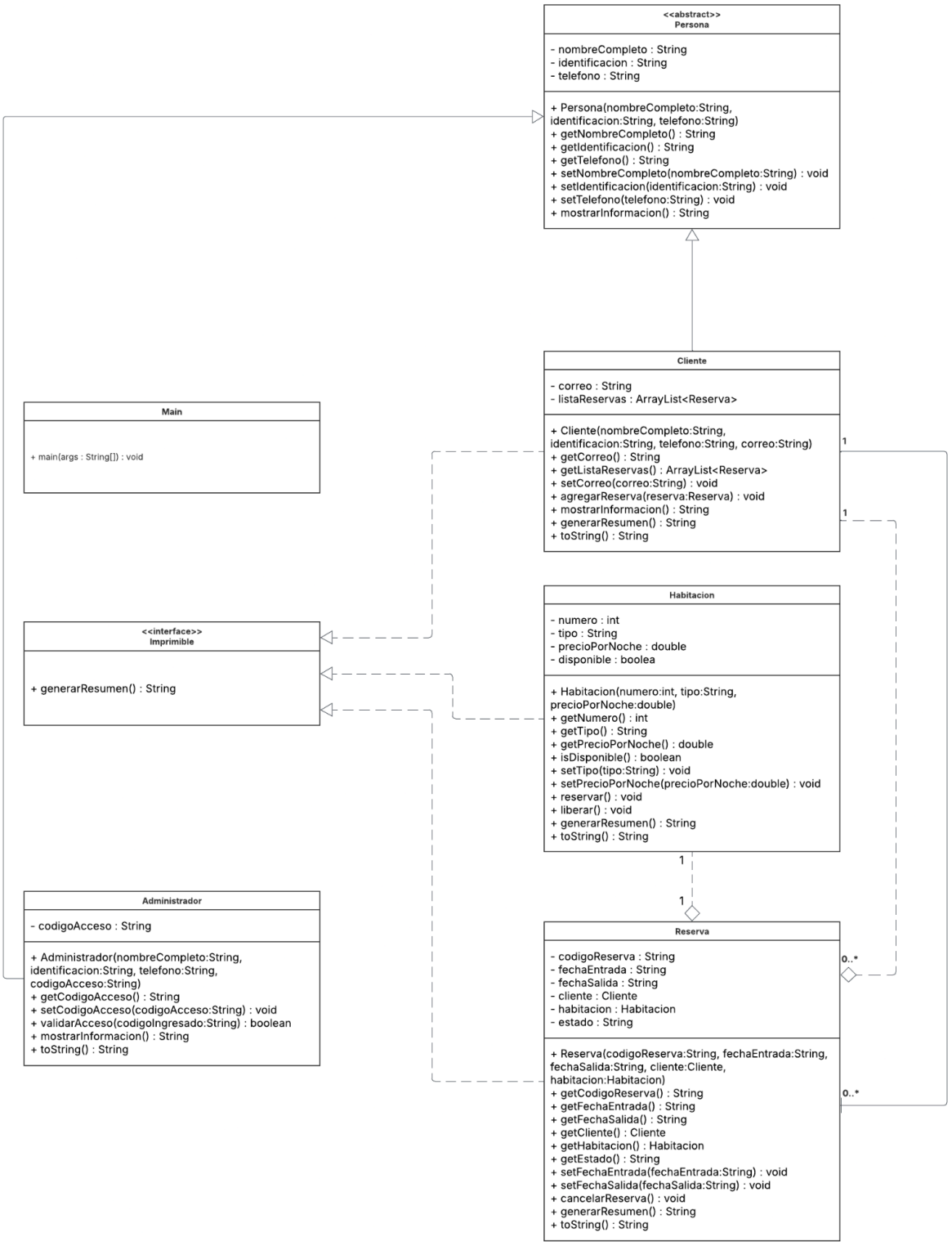
El código fuente del proyecto se encuentra disponible en el siguiente repositorio:

URL del repositorio:

https://github.com/MrChow1128/AVANCE-DEL-PORTAFOLIO-DE-EVIDENCIAS

Diagrama UML

A continuación se presenta el diagrama UML del sistema, el cual muestra la estructura de clases, sus atributos, métodos y relaciones.



Este diagrama representa:

- Herencia entre Persona, Cliente y Administrador
- Implementación de la interfaz Imprimible
- Relación de agregación entre Cliente y Reserva
- Relación de composición entre Reserva y Habitacion

El UML permite visualizar claramente la organización del sistema y cómo interactúan sus componentes.

Conclusión

El desarrollo de este proyecto permitió aplicar de manera práctica los conceptos fundamentales de la programación orientada a objetos en un sistema funcional. A lo largo del proceso se logró integrar principios como abstracción, encapsulamiento, herencia e interfaces, junto con el uso de arquitectura por capas y persistencia en base de datos.

Uno de los principales retos fue la configuración de la conexión con MySQL y la correcta implementación del patrón DAO, lo cual permitió comprender mejor la separación entre lógica de negocio y acceso a datos. Además, el manejo de excepciones personalizadas contribuyó a mejorar la robustez y control de errores dentro del sistema.

En general, este proyecto refuerza el entendimiento de cómo estructurar aplicaciones reales, preparando para desarrollos más complejos en el ámbito profesional.